

# p0019R8

## Atomic Ref

Published Proposal, 7 June 2018

### This version:

<https://github.com/ORNL/cpp-proposals-pub/blob/master/P0019/P0019r7.html>

### Authors:

[Daniel Sunderland](#)  
[H. Carter Edwards](#)  
[Hans Boehm](#)  
[Olivier Giroux](#)  
[Mark Hoemmen](#)  
[David Hollman](#)  
[Bryce Adelstein Lelbach](#)  
[Jens Maurer](#)

### Audience:

LWG

### Toggle Diffs:

☐ Hide deleted text

### Project:

ISO JTC1/SC22/WG21: Programming Language C++

---

## Abstract

Extension to the atomic operations library to allow atomic operations to apply to non-atomic objects.

## Table of Contents

|     |                                                                                     |
|-----|-------------------------------------------------------------------------------------|
| 1   | <b>Revision History</b>                                                             |
| 2   | <b>Overview</b>                                                                     |
| 3   | <b>Motivation</b>                                                                   |
| 3.1 | Atomic Operations on a Single Non-atomic Object                                     |
| 3.2 | Atomic Operations on Members of a Very Large Array                                  |
| 4   | <b><i>Reference-ability Constraints</i></b>                                         |
| 5   | <b>Concern with <code>atomic&lt;T&gt;</code> and padding bits in <code>T</code></b> |
| 6   | <b>Naming <i>atomic reference of T</i></b>                                          |
| 7   | <b>Future Work</b>                                                                  |
| 8   | <b>Proposal</b>                                                                     |
| 9   | <b>Feature Testing</b>                                                              |
|     | <b>References</b>                                                                   |
|     | Informative References                                                              |

[P0019r8]

- [2018-06-04 Rapperswil SG1](#)

SG1 straw poll to forward to LWG for inclusion in IS20

| SF | F  | N | A | SA |
|----|----|---|---|----|
| 11 | 14 | 2 | 1 | 1  |

SA: Reported that the ThreadSanitizer team potentially opposed this paper

- 2018-06-05 ThreadSanitizer team is not opposed, SA would now vote N.

A: Believes that a TS is a better ship vehicle.

- 2018-06-05 SG1

Concern about `atomic_ref` of something that is not always lock free

Consensus for no change. No objections.

- [2018-06-05 LWG](#)

Many editorial changes requested.

[P0019r7]

- [2018-03-15 Jacksonville LWG review](#)
- Update to reflect Jacksonville LWG review
- Update to reference resolution of padding bits from [P0528r2]
- Add a note clarifying that `atomic_ref` might not be lock free even if `atomic` is lock free
- Add wording for all member functions and specializations (in previous version only the constructor had wording)
- Added reference implementation
- targeted towards IS20
- Convert to bikeshed

[\[P0019r6\]](#)

- [2017-11-07 Albuquerque LEWG review](#)
  - Settle on name `atomic_ref`
  - Split out `atomic_ref<T[]>` into a separate paper, apply editorial changes accordingly
  - Restore copy constructor; not assignment operator
  - add **Throws: Nothing** to constructor but do not add noexcept
  - Remove *wrapping* terminology
  - Address problem of CAS on `atomic_ref<T>` where `T` is a struct containing padding bits
  - With these revisions move to LWG

[\[P0019r5\]](#)

- 2017-03-01 Kona LEWG review
  - Merge in P0440 Floating Point Atomic View because LEWG consensus to move P0020 Floating Point Atomic to C++20 IS
  - Rename from `atomic_view` and `atomic_array_view`; authors' selection `atomic_ref<T>` and `atomic_ref<T[]>`, other name suggested `atomic_wrapper`.
  - Remove `constexpr` qualification from default constructor because this qualification constrains implementations and does not add apparent value.

- Remove default constructor, copy constructor, and assignment operator for tighter alignment with `atomic<T>` and prevent empty references.
- Revise syntax to align with [\[P0558r1\]](#), Resolving atomic base class inconsistencies
- Recommend feature next macro

#### [\[P0019r4\]](#)

- wrapper constructor strengthen requires clause and omit throws clause
- Note types must be trivially copyable, as required for all atomics
- 2016-11-09 Issaquah SG1 decision: move to LEWG targeting Concurrency TS V2

#### [\[P0019r3\]](#)

- Align proposal with content of corresponding sections in N5131, 2016-07-15.
- Remove the *one root wrapping constructor* requirement from `atomic_array_view`.
- Other minor revisions responding to feedback from SG1 @ Oulu.

## § 2. Overview

This paper proposes an extension to the atomic operations library [**atomics**] to allow atomic operations to apply to non-atomic objects. As required by [**atomics.types.generic**] the value type **T** must be trivially copyable.

This paper includes *atomic floating point* capability defined in [\[P0020r5\]](#).

*Note:* A reference implementation ([https://github.com/ORNL/cpp-proposals-pub/blob/master/P0019/atomic\\_ref.hpp](https://github.com/ORNL/cpp-proposals-pub/blob/master/P0019/atomic_ref.hpp)) is available that works on compilers which support the [GNU atomic builtin](#) functions including recent versions of g++, icpc, and clang++. --end note

This paper is currently targeting C++20.

## § 3. Motivation

### § 3.1. Atomic Operations on a Single Non-atomic Object

An *atomic reference* is used to perform atomic operations on a referenced non-atomic object. The intent is for *atomic reference* to provide the best-performing implementation of atomic operations for the non-atomic object type. All atomic operations performed through an *atomic reference* on a referenced non-atomic object are atomic with respect to any other *atomic reference* that references the same object, as defined by equality of pointers to that object. The intent is for atomic operations to directly update the referenced object. An *atomic reference constructor* might acquire a resource, such as a lock from a collection of address-sharded locks, to perform atomic operations. Such *atomic reference* objects are not lock free and not address free. When such a resource is necessary, subsequent copy and move constructors and assignment operators might reduce overhead by copying or moving the previously acquired resource as opposed to re-acquiring that resource.

Introducing concurrency within legacy code might require replacing operations on existing non-atomic objects with atomic operations such that the non-atomic object cannot be replaced with an *atomic* object.

An object could be heavily used non-atomically in well-defined phases of an application. Forcing such objects to be exclusively *atomic* would incur an unnecessary performance penalty.

### § 3.2. Atomic Operations on Members of a Very Large Array

High-performance computing (HPC) applications use very large arrays. Computations with these arrays typically have distinct phases that allocate and initialize members of the array, update members of the array, and read members of the array. Parallel algorithms for initialization (e.g., zero fill) have non-conflicting access when assigning member values. Parallel algorithms for updates have conflicting access

to members which must be guarded by atomic operations. Parallel algorithms with read-only access require best-performing streaming read access, random read access, vectorization, or other guaranteed non-conflicting HPC pattern.

## § 4. Reference-ability Constraints

An object referenced by an *atomic reference* must satisfy possibly architecture-specific constraints. For example, the object might need to be properly aligned in memory or might not be allowed to reside in GPU register memory. We do not enumerate all potential constraints or specify behavior when these constraints are violated. It is a quality-of-implementation issue to generate appropriate information when constraints are violated.

*Note:* Whether an implementation of `atomic<T>` is lock free, does not necessarily constrain whether the corresponding implementation of `atomic_ref<T>` is lock free. --end note

## § 5. Concern with `atomic<T>` and padding bits in `T`

A concern has been discussed for `atomic<T>` where `T` is a class type that contains padding bits and how construction and `compare_exchange` operations are effected by the value of those padding bits. We require that the resolution of padding bits follow [P0528r2].

## § 6. Naming *atomic reference* of `T`

2018-03-15 Suggested that *atomic reference* of `T` be named `atomic_ref<T>` instead of `atomic<T&>`?

Result of SG1 poll

| SF | F | N  | A | SA |
|----|---|----|---|----|
| 0  | 7 | 11 | 3 | 1  |

Those against `atomic<T&>` raised the concern that it allows dangerous errors to creep into generic code, which requires users to be aware of this edge case to avoid. Also, after an `atomic<T>` is constructed it does not have data races with other objects, while an *atomic reference* of `T` does. Furthermore `atomic<T&>` does not have *volatile* member functions. Consequently, `atomic<T&>` is a specialization of `atomic<T>` with weaker guarantees.

The arguments for `atomic<T&>` is that it is more concise and reduces the vocabulary terms that a user needs to know.

We decided to keep the name of an *atomic reference* of `T` as `atomic_ref<T>` for two reasons. First, using the name `atomic_ref<T>` removes any possibility of impacting existing generic code which uses `atomic<T>`. Second, when trying to create wording for `atomic<T&>` the specializations had a distinct `vector<bool>` feel where each specialization needed to walk back from guarantees made by the primary template. In particular, the `atomic<T&>` specializations would be unable to use the phrase "*Descriptions are provided below only for members that differ from the primary template*".

## § 7. Future Work

1. Rewrite `atomic<T>` in terms of `atomic_ref<T>`?

Result of SG1 poll

| SF | F | N  | A | SA |
|----|---|----|---|----|
| 4  | 3 | 13 | 0 | 0  |

[P0019r8] duplicates much of the wording of `atomic<T>` and it is desirable to reduce this overhead. However, rewriting `atomic<T>` would require revisiting the contentious issue of `atomic<T>` having

- an exposition only member of type `T`. While there is strong support for this rewrite in SG1 and LWG, we are limiting the scope of this paper to *atomic reference* to reduce unnecessary conflicts.
2. The free functions and macros intended for C compatibility are deliberately omitted from this paper, but can be added if/when a need for them arises.

## § 8. Proposal

The proposed changes are relative to the working draft of the standard as of [\[N4727\]](#).

Text in blockquotes is not proposed wording

The `◆` character is used to denote a placeholder section number which the editor shall determine.

[ *Note*: Most of the wording in this paper is duplicated from the wording for `atomic<T>`. A future paper is needed to define `atomic<T>` in terms of `atomic_ref<T>` and improve the current wording for *atomic objects*. --end note]

Apply the following changes to 32.2.◆ [atomics.syn]:

```
namespace std {  
  
    // 3.◆ atomic ref  
    template<class T> struct atomic_ref;  
    // 3.◆ atomic ref partial specialization for pointers  
    template<class T> struct atomic_ref<T*>;  
  
}
```

Add a new subsection [atomics.ref.generic] before [atomics.types.generic]

**Class template** `atomic_ref`

```

template<class T> struct atomic_ref {
private:
    T* ptr; // exposition only
public:
    using value_type = T;
    static constexpr bool is_always_lock_free = implementation-defined;
    static constexpr size_t required_alignment = implementation-defined;

    atomic_ref() = delete;
    atomic_ref& operator=(const atomic_ref&) = delete;

    explicit atomic_ref(T&);
    atomic_ref(const atomic_ref&) noexcept;

    T operator=(T) const noexcept;
    operator T() const noexcept;

    bool is_lock_free() const noexcept;
    void store(T, memory_order = memory_order_seq_cst) const noexcept;
    T load(memory_order = memory_order_seq_cst) const noexcept;
    T exchange(T, memory_order = memory_order_seq_cst) const noexcept;
    bool compare_exchange_weak(T&, T,
                               memory_order, memory_order) const noexcept;
    bool compare_exchange_strong(T&, T,
                                  memory_order, memory_order) const noexcept;
    bool compare_exchange_weak(T&, T,
                               memory_order = memory_order_seq_cst) const noexcept;
    bool compare_exchange_strong(T&, T,
                                  memory_order = memory_order_seq_cst) const noexcept;
};

```

An `atomic_ref` object applies atomic operations [atomics.general] to the object referenced by `*ptr` such that, for the lifetime [basic.life] of the `atomic_ref` object, the object referenced by `*ptr` is an atomic object [intro.races].

The template argument for `T` shall be trivially copyable [basic.types].

The lifetime [basic.life] of an object referenced by `*ptr` shall exceed the lifetime of all `atomic_refs` that reference the object. While any `atomic_ref` instances exist which reference the `*ptr` object all accesses to that object shall exclusively occur through those `atomic_ref` instances. No subobject of the object referenced by `atomic_ref` shall be concurrently referenced by any other `atomic_ref` object.

Atomic operations applied to an object through a referencing `atomic_ref` are atomic with respect to atomic operations applied through any other `atomic_ref` referencing the same object.

[Note: Atomic operations or the `atomic_ref` constructor could acquire a shared resource, such as a lock associated with the referenced object, to enable atomic operations applied to the referenced object. - end note]

Add a new subsection [atomics.ref.operations] after [atomics.ref.generic]

## Operations on atomic types

**static constexpr bool is\_always\_lockfree;**

The static data member `is_always_lock_free` is `true` if the `atomic_ref` type's operations are always lock-free, and `false` otherwise.

**static constexpr size\_t required\_alignment;**

The required alignment of an object to be referenced by an atomic reference, which is at least `alignof(T)`.

[Note: Hardware could require that an object to be referenced by an `atomic_ref` have stricter alignment [basic.align] than other objects of type `T`. Further, whether operations on an `atomic_ref` are lock-free

could depend on the alignment of the referenced object. For example, lock-free operations on `std::complex<double>` could be supported only if aligned to `2*alignof(double)`. - end note ]

**atomic\_ref(T& obj);**

*Requires:* The referenced object shall be aligned to `required_alignment`.

*Effects:* Constructs an atomic reference that references the object.

*Throws:* Nothing.

**atomic\_ref(const atomic\_ref& ref) noexcept;**

*Effects:* Construct an atomic reference that references the object referenced by `ref`.

**T operator=(T desired) const noexcept;**

*Effects:* Equivalent to:

```
store(desired);  
return desired;
```

**operator T() const noexcept;**

*Effects:* Equivalent to: `return load();`

**bool is\_lock\_free() const noexcept;**

*Returns:* `true` if the object's operations are lock-free, `false` otherwise.

**void store(T desired, memory\_order order = memory\_order\_seq\_cst) const noexcept;**

*Requires:* The `order` argument shall not be `memory_order_consume`, `memory_order_acquire`, nor `memory_order_acq_rel`.

*Effects:* Atomically replaces the value referenced by `*ptr` with the value of `desired`. Memory is affected according to the value of `order`.

**T load(memory\_order order = memory\_order\_seq\_cst) const noexcept;**

*Requires:* The `order` argument shall not be `memory_order_release` nor `memory_order_acq_rel`.

*Effects:* Memory is affected according to the value of `order`.

*Returns:* Atomically returns the value referenced by `*ptr`.

**T exchange(T desired, memory\_order order = memory\_order\_seq\_cst) noexcept;**

*Effects:* Atomically replaces the value referenced by `*ptr` with `desired`. Memory is affected according to the value of `order`. These operations are atomic read-modify-write operations [intro.multithread].

*Returns:* Atomically returns the value referenced by `*ptr` immediately before the effects.

**bool compare\_exchange\_weak(T& expected, T desired, memory\_order success, memory\_order failure) const noexcept;**

**bool compare\_exchange\_strong(T& expected, T desired, memory\_order success, memory\_order failure) const noexcept;**

**bool compare\_exchange\_weak(T& expected, T desired, memory\_order order = memory\_order\_seq\_cst) const noexcept;**

**bool compare\_exchange\_strong(T& expected, T desired, memory\_order order = memory\_order\_seq\_cst) const noexcept;**

*Requires:* The failure argument shall not be `memory_order_release` nor `memory_order_acq_rel`.

*Effects:* Retrieves the value in `expected`. It then atomically compares the value referenced by `*ptr` for equality with that previously retrieved from `expected`, and if `true`, replaces the value referenced by `*ptr` with that in `desired`. If and only if the comparison is `true`, memory is affected according to the value of `success`, and if the comparison is `false`, memory is affected according to the value of `failure`. When only one `memory_order` argument is supplied, the value of `success` is `order`, and the value of `failure` is `order` except that a value of `memory_order_acq_rel` shall be replaced by the value `memory_order_acquire` and a value of `memory_order_release` shall be replaced by the value `memory_order_relaxed`. If and only if the comparison is `false` then, after the atomic operation, the contents of the memory in `expected` are replaced by the value read from the value referenced by `*ptr` during the atomic comparison. If the operation returns `true`, these operations are atomic read-modify-write operations [intro.races] on the value referenced by `*ptr`. Otherwise, these operations are atomic load operations on that memory.

*Returns:* The result of the comparison.

*Remarks:* A weak compare-and-exchange operation may fail spuriously. That is, even when the contents of memory referred to by `expected` and `ptr` are equal, it may return `false` and store back to `expected` the same memory contents that were originally there. [ *Note:* This spurious failure enables implementation of compare-and-exchange on a broader class of machines, e.g., load-locked store-conditional machines. A consequence of spurious failure is that nearly all uses of weak compare-and-exchange will be in a loop. When a compare-and-exchange is in a loop, the weak version will yield better performance on some platforms. When a weak compare-and-exchange would require a loop and a strong one would not, the strong one is preferable. — end note ]

Add a new subsubsection [atomics.ref.int] following the [atomics.ref.operations] subsubsection

### Specializations for integral types

There are specializations of the atomic template for the integral types `char`, `signed char`, `unsigned char`, `short`, `unsigned short`, `int`, `unsigned int`, `long`, `unsigned long`, `long long`, `unsigned long long`, `char16_t`, `char32_t`, `wchar_t`, and any other types needed by the typedefs in the header `<cstdint>`. For each such integral type *integral*, the specialization `atomic_ref<integral>` provides additional atomic operations appropriate to integral types. [ *Note:* For the specialization `atomic_ref<bool>`, see [atomics.ref.generic]. — end note ]



```

template<> struct atomic_ref<integral> {
private:
    integral* ptr; // exposition only
public:
    using value_type = integral;
    using difference_type = value_type;
    static constexpr bool is_always_lock_free = implementation-defined;
    static constexpr size_t required_alignment = implementation-defined;

    atomic_ref() = delete;
    atomic_ref& operator = (const atomic_ref&) = delete;

    explicit atomic_ref(integral&);
    atomic_ref(const atomic_ref&) noexcept;

    integral operator=(integral) const noexcept;
    operator integral () const noexcept;

    bool is_lock_free() const noexcept;
    void store(integral, memory_order = memory_order_seq_cst) const noexcept;
    integral load(memory_order = memory_order_seq_cst) const noexcept;
    integral exchange(integral,
                      memory_order = memory_order_seq_cst) const noexcept;
    bool compare_exchange_weak(integral&, integral,
                              memory_order, memory_order) const noexcept;
    bool compare_exchange_strong(integral&, integral,
                                 memory_order, memory_order) const noexcept;
    bool compare_exchange_weak(integral&, integral,
                              memory_order = memory_order_seq_cst) const noexcept;
    bool compare_exchange_strong(integral&, integral,
                                 memory_order = memory_order_seq_cst) const noexcept;

    integral fetch_add(integral,
                      memory_order = memory_order_seq_cst) const noexcept;
    integral fetch_sub(integral,
                      memory_order = memory_order_seq_cst) const noexcept;
    integral fetch_and(integral,
                      memory_order = memory_order_seq_cst) const noexcept;
    integral fetch_or(integral,
                     memory_order = memory_order_seq_cst) const noexcept;
    integral fetch_xor(integral,
                      memory_order = memory_order_seq_cst) const noexcept;

    integral operator++(int) const noexcept;
    integral operator--(int) const noexcept;
    integral operator++() const noexcept;
    integral operator--() const noexcept;
    integral operator+=(integral) const noexcept;
    integral operator-=(integral) const noexcept;
    integral operator&=(integral) const noexcept;
    integral operator|=(integral) const noexcept;
    integral operator^=(integral) const noexcept;
};

```

Descriptions are provided below only for members that differ from the primary template.

The following operations perform arithmetic computations. The key, operator, and computation correspondence are identified in Table 130 [atomics.types.int].

***integral* fetch\_key(*integral* operand, memory\_order order = memory\_order\_seq\_cst) const noexcept;**

*Effects:* Atomically replaces the value referenced by *\*ptr* with the result of the computation applied to the value referenced by *\*ptr* and the given *operand*. Memory is affected according to the value of *order*. These operations are atomic read-modify-write operations [intro.races].

*Returns:* Atomically, the value referenced by `*ptr` immediately before the effects.

*Remarks:* For signed integer types, arithmetic is defined to use two's complement representation. There are no undefined results.

**integral operator *op*=(integral operand) const noexcept;**

*Effects:* Equivalent to: `return fetch_key(operand) op operand;`

Add a new subsubsection [atomics.ref.float] following the [atomics.ref.int] subsubsection

## Specializations for floating-point types

There are specializations of the `atomic_ref` template for the floating-point types `float`, `double`, and `long double`. For each such floating-point type **floating-point**, the specialization `atomic_ref<floating-point>` provides additional atomic operations appropriate to floating-point types.

```
template<> struct atomic_ref<floating-point> {
private:
    floating-point* ptr; // exposition only
public:
    using value_type = floating-point;
    using difference_type = value_type;
    static constexpr bool is_always_lock_free = implementation-defined;
    static constexpr size_t required_alignment = implementation-defined;

    atomic_ref() = delete;
    atomic_ref& operator = (const atomic_ref&) = delete;

    explicit atomic_ref(floating-point&);
    atomic_ref(const atomic_ref&) noexcept;

    floating-point operator=(floating-point) noexcept;
    operator floating-point () const noexcept;

    bool is_lock_free() const noexcept;
    void store(floating-point, memory_order = memory_order_seq_cst) const noexcept;
    floating-point load(memory_order = memory_order_seq_cst) const noexcept;
    floating-point exchange(floating-point,
                           memory_order = memory_order_seq_cst) const noexcept;
    bool compare_exchange_weak(floating-point&, floating-point,
                              memory_order, memory_order) const noexcept;
    bool compare_exchange_strong(floating-point&, floating-point,
                                memory_order, memory_order) const noexcept;
    bool compare_exchange_weak(floating-point&, floating-point,
                              memory_order = memory_order_seq_cst) const noexcept;
    bool compare_exchange_strong(floating-point&, floating-point,
                                memory_order = memory_order_seq_cst) const noexcept;

    floating-point fetch_add(floating-point,
                            memory_order = memory_order_seq_cst) const noexcept;
    floating-point fetch_sub(floating-point,
                            memory_order = memory_order_seq_cst) const noexcept;

    floating-point operator+=(floating-point) const noexcept;
    floating-point operator-=(floating-point) const noexcept;
};
```

Descriptions are provided below only for members that differ from the primary template.

The following operations perform arithmetic computations. The key, operator, and computation correspondence are identified in Table 130 [atomics.types.int].

**floating-point fetch\_key(floating-point operand, memory\_order order = memory\_order\_seq\_cst) const noexcept;**

**Effects:** Atomically replaces the value referenced by `*ptr` with the result of the computation applied to the value referenced by `*ptr` and the given `operand`. Memory is affected according to the value of `order`. These operations are atomic read-modify-write operations [intro.races].

**Returns:** Atomically, the value referenced by `*ptr` immediately before the effects.

**Remarks:** If the result is not a representable value for its type [expr.pre] the result is unspecified, but the operations otherwise have no undefined behavior. Atomic arithmetic operations on floating-point should conform to the `std::numeric_limits<floating-point>` traits associated with the *floating-point* type [limits.syn]. The floating-point environment [cfenv] for atomic arithmetic operations on floating-point may be different than the calling thread's floating-point environment.

**floating-point operator `op=(floating-point operand) const noexcept`;**

**Effects:** Equivalent to: `return fetch_key(operand) op operand;`

Add a new subsubsection [atomics.ref.pointer] following the [atomics.ref.float] subsubsection

## Partial specialization for pointers

```
template<class T> struct atomic_ref<T*> {
private:
    T** ptr; // exposition only
public:
    using value_type = T*;
    using difference_type = ptrdiff_t;
    static constexpr bool is_always_lock_free = implementation-defined;
    static constexpr size_t required_alignment = implementation-defined;

    atomic_ref() = delete;
    atomic_ref& operator = (const atomic_ref&) = delete;

    explicit atomic_ref(T*&);
    atomic_ref(const atomic_ref&) noexcept;

    T* operator=(T*) const noexcept;
    operator T* () const noexcept;

    bool is_lock_free() const noexcept;
    void store(T*, memory_order = memory_order_seq_cst) const noexcept;
    T* load(memory_order = memory_order_seq_cst) const noexcept;
    T* exchange(T*, memory_order = memory_order_seq_cst) const noexcept;
    bool compare_exchange_weak(T*&, T*,
                              memory_order, memory_order) const noexcept;
    bool compare_exchange_strong(T*&, T*,
                                 memory_order, memory_order) const noexcept;
    bool compare_exchange_weak(T*&, T*,
                              memory_order = memory_order_seq_cst) const noexcept;
    bool compare_exchange_strong(T*&, T*,
                                 memory_order = memory_order_seq_cst) const noexcept;

    T* fetch_add(difference_type, memory_order = memory_order_seq_cst) const noexcept;
    T* fetch_sub(difference_type, memory_order = memory_order_seq_cst) const noexcept;

    T* operator++(int) const noexcept;
    T* operator--(int) const noexcept;
    T* operator++() const noexcept;
    T* operator--() const noexcept;
    T* operator+=(difference_type) const noexcept;
    T* operator-=(difference_type) const noexcept;
};
```

Descriptions are provided below only for members that differ from the primary template.

The following operations perform arithmetic computations. The key, operator, and computation correspondence are identified in Table 130 [atomics.types.pointer].

**T\* fetch\_key(difference\_type operand, memory\_order order = memory\_order\_seq\_cst) const noexcept;**

*Requires:* T shall be an object type, otherwise the program is ill-formed

*Effects:* Atomically replaces the value referenced by \*ptr with the result of the computation applied to the value referenced by \*ptr and the given operand. Memory is affected according to the value of order. These operations are atomic read-modify-write operations [intro.races].

*Returns:* Atomically, the value referenced by \*ptr immediately before the effects.

*Remarks:* The result may be an undefined address, but the operations otherwise have no undefined behavior.

**T\* operator op=(difference\_type operand) const noexcept;**

*Effects:* Equivalent to: `return fetch_key(operand) op operand;`

Add a new subsection [atomics.ref.memberops] following the [atomics.ref.pointer] subsection

**Member operators of atomic\_ref common to integers and pointers to objects**

**T\* operator++(int) const noexcept;**

*Effects:* Equivalent to: `return fetch_add(1);`

**T\* operator--(int) const noexcept;**

*Effects:* Equivalent to: `return fetch_sub(1);`

**T\* operator++() const noexcept;**

*Effects:* Equivalent to: `return fetch_add(1) + 1;`

**T\* operator--(int) const noexcept;**

*Effects:* Equivalent to: `return fetch_sub(1) - 1;`

## § 9. Feature Testing

The `__cpp_lib_atomic_ref` feature test macro should be added.

## § References

### § Informative References

[N4727]

Richard Smith. [Working Draft, Standard for Programming Language C++](https://wg21.link/n4727). URL: <https://wg21.link/n4727>

[P0019r3]

H. Carter Edwards, Hans Boehm, Olivier Giroux, James Reus. [Atomic View](https://wg21.link/p0019r3). 14 October 2016. URL: <https://wg21.link/p0019r3>

[P0019r4]

H. Carter Edwards, Hans Boehm, Olivier Giroux, James Reus. [Atomic View](https://wg21.link/p0019r4). 9 November 2016. URL: <https://wg21.link/p0019r4>

[P0019r5]

H. Carter Edwards, Hans Boehm, Olivier Giroux, James Reus. [Atomic View](https://wg21.link/p0019r5). URL: <https://wg21.link/p0019r5>

[P0019r6]

H. Carter Edwards, Hans Boehm, Olivier Giroux, James Reus. [Atomic View](https://wg21.link/p0019r6). URL:  
<https://wg21.link/p0019r6>

**[P0020r5]**

H. Carter Edwards, Hans Boehm, Olivier Giroux, JF Bastien, James Reus. [Floating Point Atomic](https://wg21.link/p0020r5). URL:  
<https://wg21.link/p0020r5>

**[P0558r1]**

[Resolving atomic named base class inconsistencies](https://wg21.link/p0558r1). URL: <https://wg21.link/p0558r1>